AUTHENTICATION REACT DEMO · CLIENT/SRC/LIB/AXIOS.TS

AXIOS INTERCEPTORS

How M11 silently refreshes an expired token and retries — a visual companion to AXIOS-INTERCEPTORS-EXPLAINED.md

1. The instance → 2. Skip list → 3. Dedup refresh → 4. Error handler → 5. Retry or redirect → 6. Full sequence

1 What an interceptor actually is

An axios interceptor is just **a function that runs automatically on every request or every response** — before your own `.then()`/`await` code ever sees it. Think of it as a checkpoint every API call has to pass through: a place to inject shared logic ("if this failed with a 401, try to fix that before giving up") once, instead of repeating it in every hook that makes a request.

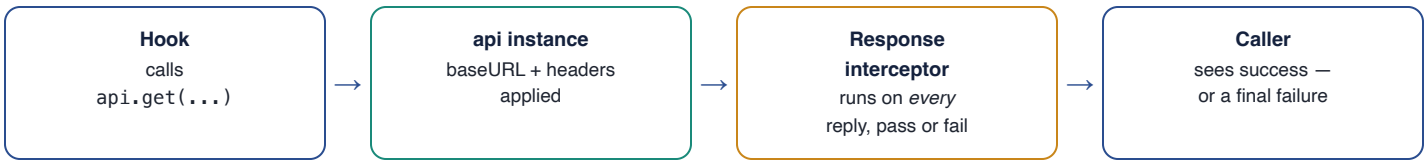
2 The instance itself

client/src/lib/axios.ts

```
export const api = axios.create({
  baseURL: "/api",
  headers: { "Content-Type": "application/json" },
});
```

- `axios.create({...})` — a *new, independent* instance with its own settings, instead of the global `axios` object. The interceptor below only applies to `api` — not to every axios call anywhere in the app. That distinction matters later, for the refresh call itself.
- `baseURL: "/api"` — every call site writes `api.get("/consumers")` instead of `api.get("/api/consumers")`; axios prepends it automatically.
- `headers: {...}` — set once here instead of repeated in every call.

→ Where this sits in the request path



3 Deciding which failures are worth retrying

client/src/lib/axios.ts

```
const AUTH_ENDPOINTS_TO_SKIP = ["/auth/login", "/auth/signup", "/auth/refresh"];

function shouldAttemptRefresh(config) {
  if (!config?.url) return false;
  return !AUTH_ENDPOINTS_TO_SKIP.some(path => config.url.includes(path));
}
```

- `/auth/login` 401 = wrong password — retrying via refresh makes no sense, there's no session to refresh yet.
- `/auth/signup` 401 can't actually happen (signup doesn't require auth), but it's excluded defensively anyway.
- `/auth/refresh` 401 = the refresh token itself is dead. Letting *this* trigger another refresh attempt, and that also 401s, and it tries again... is an infinite loop.
- `.some(...)` checks if the URL contains *any* skip-list path; the leading `!` inverts it — "attempt a refresh" only when the URL matches **none** of them.

4 Avoiding duplicate refresh calls

client/src/lib/axios.ts

```
let refreshPromise = null;

function refreshAccessToken() {
  if (!refreshPromise) {
    refreshPromise = axios.post("/api/auth/refresh")
      .then(() => undefined)
      .finally(() => { refreshPromise = null; });
  }
  return refreshPromise;
}
```

The problem this solves: a page firing 3 API calls at once, all hitting an expired token, would otherwise trigger 3 independent `POST /auth/refresh` calls — wasteful, and racy if the refreshes overlap.

- `let refreshPromise` — lives at module scope, shared by every request that goes through this file, persisting between calls.
- Plain `axios.post(...)`, **not** the `api` instance — using `api` here would make this exact refresh call subject to its own interceptor: recursive risk.
- `.finally(() => refreshPromise = null)` — resets after the refresh settles (success or failure) so the *next* 401, later, starts a genuinely fresh attempt.
- Every caller during the same in-flight window gets the exact same `Promise` back — one network call, shared result.

5 Success vs. failure handlers

```
api.interceptors.response.use(
  (response) => response,              // 2xx — pass through, untouched
  async (error) => { /* recovery logic */ }  // everything else
);
```

6 Three conditions — any ONE stops the retry

1. **Not a 401**
   500 / network error / 404 — refreshing a token won't fix a server crash
2. **Already retried**
   `originalRequest._retry` is set — tried once, still failed, stop
3. **Excluded endpoint**
   `shouldAttemptRefresh()` says no — login/signup/refresh itself

```
if (!isUnauthorized || alreadyRetried || !shouldAttemptRefresh(originalRequest)) {
  return Promise.reject(error); // re-throw — caller's own catch runs normally
}
```

7 Otherwise: retry once, or redirect

```
try {
  originalRequest._retry = true;            // never retry the SAME request twice
  await refreshAccessToken();              // throws if refresh token is also dead
  return api(originalRequest);             // replay with the new cookie
} catch (refreshError) {
  window.location.href = "/login";    // hard redirect — no useNavigate() here
  return Promise.reject(refreshError);
}
```

- `_retry = true` — marks this request so if the retry *also* 401s, condition 2 above catches it instead of looping forever.
- `api(originalRequest)` replays the exact original request (same URL, method, body) — the caller receives this as a normal success, unaware a refresh ever happened.
- `window.location.href`, not React Router's `navigate()` — this code runs outside any component, so a hard reload is the simplest clean-logout path.

8 The complete flow

1 — NORMAL CALL

`Component → api.get("/consumers", ...)`

2 · 3 — TOKEN HAS EXPIRED

`access_token cookie expired → server responds 401 → response interceptor's error handler fires`

Checks: real 401? not already retried? not an excluded auth endpoint? All yes → proceed.

4 · 5 · 6 — SILENT RECOVERY

`refreshAccessToken() → POST /auth/refresh` (deduplicated if several requests hit this at once)

`server verifies refresh cookie → mints new access-token cookie → Set-Cookie, browser stores it`

`interceptor replays GET /consumers → succeeds with the fresh cookie`

7 — CALLER IS NONE THE WISER

`original caller` receives a normal, successful response — `no idea a 401 and a refresh happened in between`

8 — THE ESCAPE HATCH

`IF step 4 had failed instead` (refresh token also dead) `→ window.location.href = "/login"`

Proven live (M11, AUTH-LEARNING-MILESTONES.md): a request made with an already-expired access token succeeded transparently — the calling code never saw the failure. See also `AXIOS-MIGRATION-EXPLAINED.md` for every hook and component that actually calls through this engine.